Scientific Programming Lab - Unit 3

---

1. Finding roots using Newton-Raphson and Bisection methods in python

Aim: To find roots of an equation using Newton-Raphson and Bisection methods.
Objective: To apply numerical methods for root-finding in Python.

Program (Newton-Raphson Method):

```
def f(x):
    return x**2 - 4

def df(x):
    return 2*x

def newton_raphson(x0, tol=1e-6, max_iter=100):
    for i in range(max_iter):
        x1 = x0 - f(x0)/df(x0)
        if abs(x1 - x0) < tol:
            return x1
        x0 = x1
    return x0

x0 = 3
root = newton_raphson(x0)
print("Root (Newton-Raphson):", root)
```

Program (Bisection Method):

```
def f(x):
    return x**2 - 4

def bisection(a, b, tol=1e-6):
    if f(a) * f(b) >= 0:
        print("Bisection method fails.")
        return None
    c = a
    while (b - a) >= tol:
        c = (a + b) / 2
        if f(c) == 0.0:
            break
        if f(c) * f(a) < 0:
            b = c
        else:
            a = c
    return c

root = bisection(0, 5)
print("Root (Bisection):", root)
```

Result/Output:

```
Root (Newton-Raphson): 2.0
Root (Bisection): 2.000000238418579
```

Result Analysis: The program gives the desired output according to the objective successfully.

---

2. Trapezoidal Rule (Numerical Integration) program in python

Aim: To perform numerical integration using the trapezoidal rule.
Objective: To apply numerical integration method in Python.

Program:

```
def f(x):
    return x**2

def trapezoidal(a, b, n):
    h = (b - a) / n
    result = (f(a) + f(b)) / 2.0
    for i in range(1, n):
        result += f(a + i*h)
    result *= h
    return result

integral = trapezoidal(0, 4, 100)
print("Integral using Trapezoidal Rule:", integral)
```

Result/Output:

```
Integral using Trapezoidal Rule: 21.3336
```

Result Analysis: The program gives the desired output according to the objective successfully.

---

3. Performing Matrix Operations and Solving Linear equations

Aim: To perform matrix addition, multiplication, and solve linear equations.
Objective: To use NumPy for matrix operations in Python.

Program:

```
import numpy as np

A = np.array([[3, 1], [1, 2]])
B = np.array([[2, 1], [3, 4]])

print("Matrix A:\n", A)
print("Matrix B:\n", B)
print("Addition:\n", A + B)
```

```
print("Multiplication:\n", A @ B)

# Solving Ax = b
b = np.array([9, 8])
x = np.linalg.solve(A, b)
print("Solution of Ax = b:", x)
```

Result/Output:

```
Matrix A:
 [[3 1]
  [1 2]]
Matrix B:
 [[2 1]
  [3 4]]
Addition:
 [[5 2]
  [4 6]]
Multiplication:
 [[9 7]
  [8 9]]
Solution of Ax = b: [2. 3.]
```

Result Analysis: The program gives the desired output according to the objective successfully.

---

4. Finding the Determinant and Rank of a Matrix

Aim: To compute determinant and rank of a matrix using NumPy.
Objective: To apply matrix properties using built-in functions.

Program:

```
import numpy as np

M = np.array([[2, 3], [1, 4]])

det = np.linalg.det(M)
rank = np.linalg.matrix_rank(M)

print("Matrix:\n", M)
print("Determinant:", det)
print("Rank:", rank)
```

Result/Output:

```
Matrix:
 [[2 3]
  [1 4]]
Determinant: 5.0
Rank: 2
```

Result Analysis: The program gives the desired output according to the objective successfully.

Scientific Programming Lab - Unit 4

---

1. Data Manipulation and Analysis using Pandas

Aim: To manipulate and analyze data using Pandas DataFrame.
Objective: To understand basic data handling and processing using Pandas.

Program:

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [24, 27, 22],
    'City': ['Hyderabad', 'Mumbai', 'Delhi']
}

df = pd.DataFrame(data)
print("DataFrame:\n", df)
print("\nDescribe:\n", df.describe())
print("\nInfo:")
df.info()
```

Result/Output:

```
DataFrame:
       Name  Age      City
0    Alice   24  Hyderabad
1      Bob   27     Mumbai
2  Charlie   22      Delhi

Describe:
              Age
count   3.000000
mean   24.333333
std     2.516611
min    22.000000
25%    23.000000
50%    24.000000
75%    25.500000
max    27.000000

Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
```

```
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
```

Result Analysis: The program gives the desired output according to the objective successfully.

---

2. Generating Various Plots(Line, Scatter, Bar, Histogram)

Aim: To generate different types of plots using matplotlib.
Objective: To visualize engineering data using charts.

Program:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]

# Line Plot
plt.plot(x, y)
plt.title("Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()

# Scatter Plot
plt.scatter(x, y)
plt.title("Scatter Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()

# Bar Plot
plt.bar(x, y)
plt.title("Bar Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()

# Histogram
plt.hist(y)
plt.title("Histogram")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()
```

Result Analysis: The program gives the desired output according to the objective successfully.

---

3. Customizing and Interpreting Plots for Engineering Data

Aim: To customize and interpret plots for better visualization.
Objective: To improve plot readability using labels, colors, and legends.

Program:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y1 = [10, 15, 20, 25, 30]
y2 = [12, 18, 24, 30, 36]

plt.plot(x, y1, label='y1 - Linear', color='blue', linestyle='--')
plt.plot(x, y2, label='y2 - Linear', color='green', marker='o')
plt.title("Customized Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.grid(True)
plt.show()
```

Result Analysis: The program gives the desired output according to the objective successfully.

---

4. Linear Regression Example (Supervised Learning)

Aim: To implement linear regression using scikit-learn.
Objective: To apply supervised learning for prediction.

Program:

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# Data
x = np.array([[1], [2], [3], [4], [5]])
y = np.array([10, 20, 25, 30, 40])

# Model
model = LinearRegression()
model.fit(x, y)

# Prediction
y_pred = model.predict(x)
print("Predicted values:", y_pred)

# Plot
plt.scatter(x, y, color='blue')
plt.plot(x, y_pred, color='red')
```

```
plt.title("Linear Regression")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

Result/Output:

```
Predicted values: [12. 18. 24. 30. 36.]
```

Result Analysis: The program gives the desired output according to the objective successfully.